Technical Summary: Upwork Technical Support AI Bot (RAG)

Candidate Assessment Submission Document

This technical summary document details the evaluation results, engineering challenges, LLM usage, and developer suitability for the Associate AI Developer role at ProAnalyst.

1. RAG Evaluation Results (Ground Truth Queries)

The three required assignment evaluation questions were run through the Streamlit web application. The exact outputs generated by the Llama-3.1 model are documented below:

Question 1

- Query: *"What is the specific request-per-second rate limit for the Upwork API, and is it enforced per Key or per IP?"*
- LLM Generated Response:

I'm sorry, but the provided documentation does not contain that information.

- Technical Analysis: The provided document (`upwork_api_reference.txt`) contains details on GraphQL schema, OAuth authentication flows, and service account keys, but does not specify Upwork API rate limits or enforcement scopes (Key vs. IP). The Hallucination Guard successfully intercepted the query, blocking the LLM from making assumptions.

Question 2

- Query: *"How long is an OAuth access token valid for?"*
- LLM Generated Response:

According to Context Block #2, the TTL (time to live) for an access token is 24 hours.

- Technical Analysis: The vector store successfully retrieved Chunk #11 containing the text: `TTL for an access token is 24 hours; TTL for a refresh token is 2 weeks since its last usage.` The model parsed and answered the lifetime question correctly.

Question 3

- Query: *"Can I use a Client Credentials Grant to access a user's private contract details?"*
- LLM Generated Response:

I'm sorry, but the provided documentation does not contain that information.

- Technical Analysis: While Chunk #34 defines the Client Credentials Grant flow as enabling access *"outside the context of a user... to access the client's resources"*, the documentation does not explicitly detail permissions surrounding access to private contract data. To maintain high-fidelity answers and strictly adhere to context blocks, the Hallucination Guard blocked extrapolation.

2. Difficulties Faced & Engineering Resolutions

During the implementation of the RAG pipeline, the following technical challenges were encountered and

resolved:

- Managing HTTP Round-Trip Latency:
- *Challenge*: Querying a remote, hosted LLM endpoint via HTTP requests introduces round-trip network delays (typically 1.5-2.5 seconds per request). This latency impacts user interface responsiveness.
- *Resolution*: Implemented a clean visual loader in the UI, configured a strict 30-second connection timeout, and implemented dual timing statistics to track both raw API execution speed and total pipeline latency (including vector search).
- Cleaning Visual Text Artifacts from Parsing Structural GraphQL Reference Code:
- *Challenge*: Converting the technical API document from a complex PDF (`API Documentation Partial.pdf`) containing multi-column tables, cURL requests, and GraphQL fragments left raw visual artifacts, hyphenation breaks, page breaks, and fragmented brackets in the parsed text.
- *Resolution*: Implemented custom pre-processing checks during PDF text extraction, sanitized pagination lines (`--- PAGE BREAK ---`), and optimized character wrapping to preserve clean code snippets for vector embedding.
- Handling Technical Code Snippet Boundary Fragmentation:
- *Challenge*: Slicing documents purely by character counts can cut an API endpoint, cURL payload, or GraphQL query parameter in half, separating critical code structures across different vector indices.
- *Resolution*: Leveraged LangChain's `RecursiveCharacterTextSplitter` to target natural paragraph, newline, and whitespace dividers before character splitting. Configured a 50-character boundary overlap to ensure split parameters are preserved in at least one adjacent chunk context.

---

3. LLM Assistance Statement

During development, LLMs (GPT-4 / Claude 3.5 Sonnet) were utilized as pair-programming assistants to accelerate implementation:

- Boilerplate & CSS Styling: Prompted to generate modular CSS blocks for the Streamlit UI (applying glassmorphism card containers, rounded metric badges, and Google Outfit font configurations).
- Encoding Fallbacks: Assisted in designing encoding-safe stream wrapper scripts to prevent Windows terminal crashes when testing unicode characters.
- JSON Serialization: Prompted for direct python `requests` API payload formats compatible with the OpenAI API base format (`/v1/openai/chat/completions`) to minimize external library bloat.

---

4. Why I Am the Best Fit for the ProAnalyst AI Team

My engineering style and philosophy align directly with the criteria for the AI Team:

1. Defensive & Production-Ready Coding: I prioritize system robustness. When faced with Windows-specific encoding crashes, I refactored the scripts to utilize standard ASCII borders and safe byte streams rather than simple print statements, ensuring code runs flawlessly across all developer environments.

2. Archit
as the o
alignmen